UNIT I

FUNDAMENTALS OF NEURAL NETWORKS

1. Introduction:

Basic Concepts of Neural Networks

1.1. What is a Neural Network?

A Neural Network is a computational model inspired by the way biological neural systems, such as the human brain, process information. It is designed to recognize patterns and solve problems by mimicking how neurons in the brain work together.

Neural Networks consist of interconnected layers of nodes, or "neurons," where each node performs a mathematical operation. The structure typically includes:

- **Input Layer**: Receives the data.
- **Hidden Layers**: Process the data through various transformations.
- **Output Layer**: Produces the final result.

1.2. Why Neural Networks?

Neural Networks are powerful tools for:

- Image recognition (e.g., facial recognition in smartphones).
- Speech processing (e.g., virtual assistants like Siri or Alexa).
- Predictive analytics (e.g., weather forecasting).

1.3. Key Terminologies in Neural Networks

- **Neuron:** The basic unit that processes data by applying a function to inputs.
- **Weights:** Parameters that determine the importance of each input.
- **Bias:** Helps the network make decisions by adding flexibility to the activation function.
- **Activation Function:** Decides whether a neuron's output should be passed to the next layer. Examples include sigmoid, ReLU, and tanh.

Real-Life Example: Predicting House Prices

Imagine you're helping your parents decide how much to sell your house for. Several factors, such as location, size, number of bedrooms, and age of the house, influence the price.

1. **Input Layer:**
    - Location
    - Size in square feet
    - Number of bedrooms
    - Age of the house

2. **Hidden Layers:**
    - The neural network processes this information to find patterns. For example, houses in prime locations and larger houses generally have higher prices.
3. **Output Layer:**
    - Predicts the estimated price of the house.

Each factor is assigned a weight. If the house is in a good location, the network will assign a higher weight to the "location" input. Similarly, a smaller weight might be given to "age" if the house's age is less critical in determining its price.

How It Works:

The neural network adjusts the weights and biases during training (using historical data of house prices) until it accurately predicts the price for new data. This process is called **learning** or **training.**

1.4. Biological Analogy

The human brain has billions of neurons. Each neuron receives signals through dendrites, processes them, and sends the result through axons. Similarly:

- Inputs = Signals received by dendrites.
- Weights = Strength of the signal.
- Activation Function = Decides whether to "fire" the signal to the next neuron.

Real-Life Example: Spam Email Detection

When you receive an email, your inbox uses a neural network to decide whether it's spam:

- Inputs: Email features like subject line, keywords, sender's address.
- Hidden Layers: The network identifies patterns like common spam phrases.
- Output: Classifies the email as "Spam" or "Not Spam."

By training the model on millions of examples, it learns to recognize even subtle differences between spam and genuine emails.

---

2. Human Brain

2.1. Introduction to the Human Brain

The human brain is one of the most complex and powerful organs in the body. It controls all the functions of the body, processes information from the senses, and helps us think, learn, and make decisions.

The brain is made up of approximately **86 billion neurons**, which are specialized cells that transmit information. Neurons communicate with each other using electrical and chemical signals. This communication forms the foundation for everything we do, from moving our fingers to solving complex math problems.

2.2. Structure of the Human Brain

The human brain has three main parts:

1. **Cerebrum**:
    - The largest part of the brain.
    - Responsible for functions like thinking, learning, memory, and voluntary movements.
    - Divided into two hemispheres (left and right), with each controlling the opposite side of the body.
2. **Cerebellum**:
    - Located under the cerebrum.
    - Manages coordination, balance, and motor skills.
    - Helps us perform tasks like walking, running, or riding a bicycle.
3. **Brainstem**:
    - Connects the brain to the spinal cord.
    - Controls basic functions like breathing, heartbeat, and digestion.

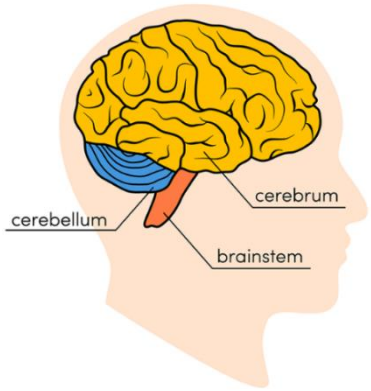


Fig: human brain

2.3. How the Brain Processes Information

The brain receives input from the environment through the **five senses**:

- **Eyes** for vision.
- **Ears** for hearing.
- **Skin** for touch.
- **Nose** for smell.
- **Tongue** for taste.

For example:

- When you see a red apple, your eyes send signals to your brain. The brain processes this visual information, identifies the apple, and even associates it with its taste or previous experiences.

2.4. Neurons: The Brain's Building Blocks

Neurons are the brain's fundamental units. They work in three stages:

1. **Input**: Neurons receive signals from the environment or other neurons.
2. **Processing**: The brain processes this information to make sense of it.
3. **Output**: The brain sends commands to the body based on the processed information.

Real-Life Example: Driving a Car

- **Input**: Your eyes see a red traffic light.
- **Processing**: Your brain recognizes the red light as a signal to stop.
- **Output**: Your brain sends signals to your foot to press the brake pedal.

2.5. Connection to Artificial Neural Networks

Artificial neural networks (ANNs) are inspired by the human brain. Similar to neurons in the brain, ANNs consist of artificial "neurons" that are connected in layers. They take input, process it, and produce output.

Real-Life Example of ANN:

- Imagine a voice assistant like Alexa.
    - **Input**: You say, "What's the weather today?"
    - **Processing**: The system processes your voice, understands the query, and checks weather data.
    - **Output**: It replies with, "Today's weather is sunny."

2.6. Why is the Human Brain Important in AI?

Scientists and engineers study the brain to build systems that mimic its capabilities. For example:

- How the brain learns helps improve **machine learning algorithms**.
- How neurons connect and communicate inspires the design of **deep learning models**.

Example: Face Recognition

Just like your brain recognizes your friend in a crowd, neural networks can identify faces using cameras and AI. The system learns patterns (like the shape of eyes and nose) to make accurate identifications.

Key Takeaways

- The human brain is the inspiration behind artificial neural networks.
- Neurons in the brain communicate using electrical and chemical signals, similar to how artificial neurons process data.
- Real-life tasks, like recognizing faces or stopping at a red light, show how the brain processes input, makes decisions, and gives outputs.

---

3. Model of Artificial Neurons

Artificial neurons are the building blocks of artificial neural networks, inspired by the biological neurons found in the human brain. To understand the model of an artificial neuron, let's break it down step by step and look at real-life examples to make it easier to grasp.

3.1. Biological Inspiration

- In the human brain, neurons are specialized cells that transmit information. Each neuron receives signals (inputs) through its dendrites, processes the information in the cell body, and then sends an output signal through its axon.
- Artificial neurons mimic this behaviour mathematically. Instead of dendrites, we have inputs; instead of processing in the cell body, we use mathematical functions; and instead of axons, we produce an output.

3.2. Components of an Artificial Neuron

An artificial neuron has the following components:

a. Inputs (▲)

Inputs are the values or data fed into the neuron. These could be numbers that represent real-world information.

- **Example**: In predicting house prices, the inputs could be factors like the number of bedrooms, size of the house, and location.

b. Weights (ω)

Each input is multiplied by a weight. A weight determines how important an input is to the overall output of the neuron.

- **Example**: In predicting house prices, location might have a higher weight than the number of bedrooms because location significantly affects the price.

c. Summation Function

All the weighted inputs are added together to produce a single value. Mathematically, this is written as:

where:

- is the input value
- is the corresponding weight
- is the summation value

d. Activation Function

The summation value is passed through an activation function, which determines whether the neuron should "fire" (produce an output) or not. Common activation functions include:

- **Sigmoid Function**: Produces outputs between 0 and 1.
- **ReLU (Rectified Linear Unit)**: Outputs 0 for negative values and the input itself for positive values.

e. Output

The result of the activation function is the output of the neuron.

- **Example**: In a spam email classifier, the output might be 1 (spam) or 0 (not spam).

3.3. Real-Life Example: Self-Driving Cars

Let’s see how an artificial neuron works in a real-life scenario:

Problem:

A self-driving car needs to decide whether to stop at a traffic signal. Inputs to the artificial neuron might include:

1. **Distance to the traffic light** (in meters)
2. **Traffic light colour** (represented as numbers: Red = 1, Yellow = 0.5, Green = 0)
3. **Speed of the car** (in km/h)

Steps:

1. **Assign Inputs:**
    - Distance: 20 meters
    - Traffic light colour: 1 (Red)
    - Speed: 50 km/h
2. **Set Weights:**
    - Distance weight: 0.3
    - Traffic light colour weight: 0.6

- Speed weight: 0.1

3. **Calculate Weighted Sum:**
4. **Apply Activation Function:** Using a threshold activation function, if , the neuron outputs 1 (stop), otherwise 0 (don't stop).
5. **Output:** Since , the output is 1, meaning the car should stop.

---

4. McCulloch-Pitts Model

The **McCulloch-Pitts model** is one of the first artificial neural network models. It was introduced in 1943 by Warren McCulloch and Walter Pitts. This model mimics the behaviour of a biological neuron in the brain.

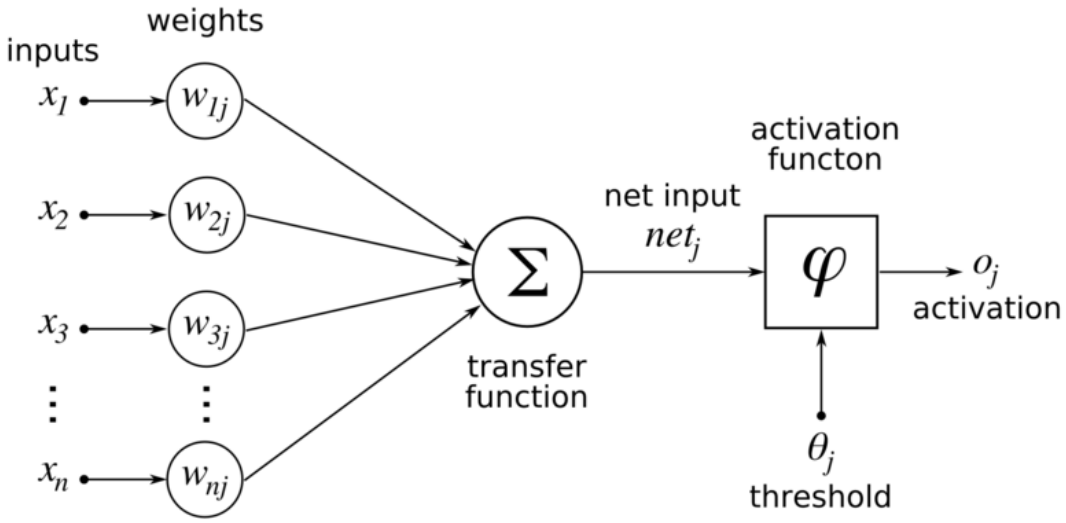
4.1. What is a Neuron?

- A **neuron** is a nerve cell in the brain that processes information and sends signals to other neurons.
- Just like our brain has billions of neurons to process thoughts, computers can use artificial neurons to solve problems.

4.2. What is the McCulloch-Pitts Neuron?

The McCulloch-Pitts neuron is a simple mathematical model of how a biological neuron works. It works in three steps:

1. **Input Signals:** It receives signals (inputs) from the outside world or other neurons.
2. **Summation:** It adds up all the input signals after multiplying them with their respective weights (importance).
3. **Threshold Function:** If the total is greater than or equal to a threshold value, it produces an output signal (like ON or OFF).



4.3. Key Components of McCulloch-Pitts Neuron

1. **Inputs (x):** These are the signals received by the neuron.
2. **Weights (w):** These are values assigned to each input to indicate its importance.
3. **Summation (Σ):** The weighted sum of all inputs.
4. **Threshold (θ):** A value that determines when the neuron "fires" or produces an output.
5. **Output (y):** The final decision (e.g., 1 for ON or 0 for OFF).

4.4. How Does It Work?

The McCulloch-Pitts neuron works as follows:

1. Multiply each input by its corresponding weight.
2. Add all the weighted inputs.
3. Compare the sum to a threshold value.
    - If the sum is greater than or equal to the threshold, the neuron outputs 1 (ON).
    - Otherwise, it outputs 0 (OFF).

4.5. Real-Life Example: Turning on a Light Bulb

Imagine you are designing a simple system to turn on a light bulb based on three conditions:

1. **Condition 1:** It's dark outside.
2. **Condition 2:** Someone is in the room.
3. **Condition 3:** The switch is turned ON.

Each condition is an **input**:

- If a condition is true, the input is 1.
- If it's false, the input is 0.

Let's assign weights to each condition:

- Condition 1 (dark): Weight = 2
- Condition 2 (presence): Weight = 3
- Condition 3 (switch): Weight = 5

Set a threshold, $\theta = 6$.

Step-by-Step Calculation

1. Check the inputs:
    - Dark (1), Someone is in the room (1), and Switch is ON (1).
      Inputs = [1, 1, 1].
2. Multiply each input by its weight:
    - (Dark × 2) + (Presence × 3) + (Switch × 5) = 2 + 3 + 5 = 10.
3. Compare the sum to the threshold:
    - $10 \geq 6$ (threshold).
4. Output:
    - The neuron "fires" (output = 1), turning on the light bulb.

4.6. Advantages of McCulloch-Pitts Model

1. **Simple Design:** Easy to understand and implement.
2. **Binary Decision:** Makes clear ON/OFF decisions like a switch.
3. **Foundation:** It laid the groundwork for modern artificial neural networks.

4.7. Limitations of McCulloch-Pitts Model

1. **Limited Learning:** It doesn't learn from experience or data.
2. **Static Weights:** The weights and threshold are fixed and must be manually assigned.
3. **No Complex Outputs:** It can only give binary outputs (1 or 0)

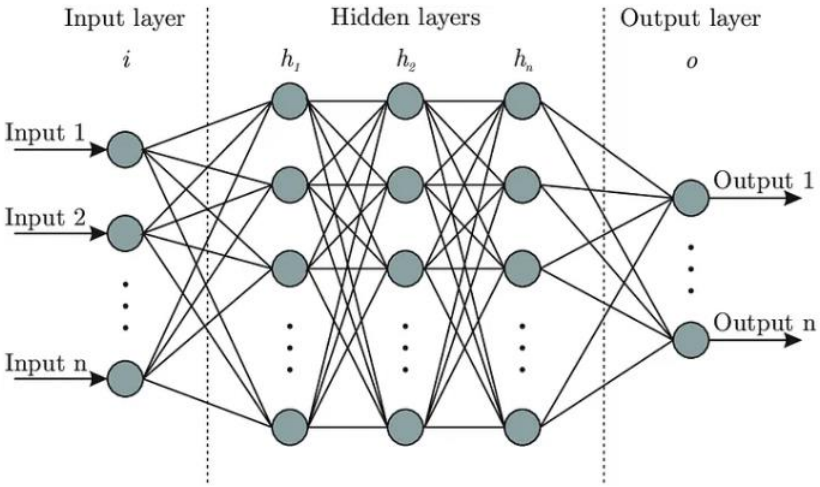
---

5. Neural Network Architecture

A neural network is a computational model inspired by the structure and functioning of the human brain. It consists of layers of interconnected nodes (neurons) designed to process information.

Key Components:

- **Input Layer:** Receives the input data.
- **Hidden Layers:** Intermediate layers where computations are performed.
- **Output Layer:** Produces the final result.
- **Weights and Biases:** Connections between neurons have weights that determine the strength of the signal, while biases adjust the output.



Example: Imagine a house pricing system. The input layer might take the size of the house, number of rooms, and location. Hidden layers process this data using mathematical operations, and the output layer predicts the price.

5.1. Characteristics of Neural Networks

Neural networks have the following key features:

1. **Non-linearity:** Neural networks can model non-linear relationships between inputs and outputs.
    - *Example:* Predicting weather based on humidity and temperature, which do not have a linear relationship.
2. **Adaptivity:** They learn from data and improve over time.
    - *Example:* A recommendation system like Netflix improving its suggestions as you watch more shows.
3. **Fault Tolerance:** Even if some neurons fail, the network can still function.
    - *Example:* A speech recognition system still works even with noisy input.
4. **Generalization:** Neural networks can generalize well to unseen data if trained properly.
    - *Example:* A handwriting recognition model accurately identifying new handwriting styles.

5.2. Learning Methods

Neural networks learn by adjusting weights and biases based on the data provided. The main learning methods are:

1. **Supervised Learning:** The model learns using labelled data, where inputs have corresponding outputs.
    - *Example:* Teaching a system to classify images of cats and dogs using a dataset with labelled images.
2. **Unsupervised Learning:** The model identifies patterns in unlabelled data.
    - *Example:* Grouping customers based on purchasing behaviour without predefined categories.
3. **Reinforcement Learning:** The model learns by interacting with its environment and receiving rewards or penalties.
    - *Example:* A self-driving car learning to navigate roads by trial and error.

5.3. Taxonomy of Neural Network Architecture

Neural networks can be classified into different types based on their structure and application:

1. **Feedforward Neural Network:** Data flows in one direction from input to output.
    - *Example:* Predicting house prices.
2. **Recurrent Neural Network (RNN):** Includes loops to process sequential data.
    - *Example:* Language translation or speech recognition.

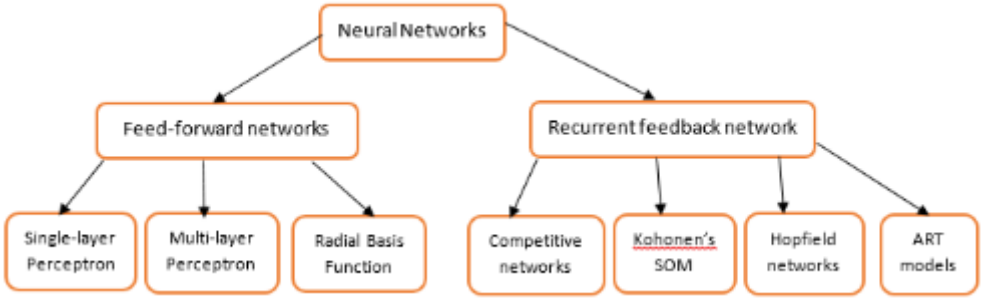


Fig: Neural Network Architecture

---

6. Early Neural Network Architectures

Early neural networks laid the foundation for modern advancements. Here are some notable ones:

6.1. Perceptron (1958)

- **Inventor:** Frank Rosenblatt
- **Architecture:** The Perceptron is one of the earliest models of a neural network and is a type of single-layer feed forward neural network. It consists of an input layer, a processing node (called the perceptron), and an output layer.
- **Function:** The perceptron performs binary classification using a linear decision boundary, where the input is multiplied by weights, summed, and passed through an activation function (usually a step function).
- **Limitations:** It can only solve linearly separable problems, like AND or OR logic gates, and fails for non-linearly separable problems (e.g., XOR).
- **Impact:** It introduced the basic structure of a neural network but faced criticism after its inability to solve XOR problems was highlighted by Marvin Minsky and Seymour Papert in their book *Perceptron's* (1969).

Example: Classifying emails as spam or not spam.

6.2. ADALINE (Adaptive Linear Neuron, 1960):

- **Inventor:** Bernard Widrow and Ted Hoff

- **Architecture:** Similar to the perceptron but with a key difference in the activation function used.
- **Function:** Instead of the step function, Adaline uses a linear activation function, and it updates weights using the Least Mean Squares (LMS) rule, minimizing the error between the predicted and actual values.

Example: Predicting stock market based on previous analysis.

6.3. Hopfield Network (1982)

- **Inventor:** John Hopfield
- **Architecture:** A recurrent neural network (RNN) with fully connected neurons that operate in an auto-associative manner.
- **Function:** It operates as a content-addressable memory system, where patterns can be stored and later retrieved from partial or noisy input.
- **Properties:** Symmetric connections between neurons, binary outputs, and the network converges to a stable state.
- **Applications:** Memory retrieval, pattern recognition, and optimization problems.
    - *Example:* Recalling a specific image from noisy data.

4. Multi-Layer Perceptron (MLP) (1980s)

- **Inventors:** Various researchers (notably including Paul Werbos and Geoffrey Hinton)
- **Architecture:** A more advanced version of the perceptron, consisting of multiple layers of neurons (input layer, hidden layers, and output layer).
- **Function:** Uses backpropagation, a key algorithm that allows MLP to train more complex, non-linear models by adjusting the weights based on the gradient of the error.
- **Activation Functions:** Sigmoid, tanh, and later ReLU and Softmax.
- **Impact:** The introduction of multi-layer networks with backpropagation made it possible to train neural networks on complex tasks, such as image and speech recognition.
    - *Example:* Detecting fraudulent transactions.

---

7. Perceptron Model

The perceptron is one of the simplest types of artificial neural networks. It was introduced by Frank Rosenblatt in 1958 and forms the foundation for more complex neural network models.

7.1. What is a Perceptron?

A perceptron is a computational model inspired by the way biological neurons work. It takes multiple input values, applies weights to them, sums them up, and then passes the result through an activation function to produce an output.

The perceptron can only solve problems that are linearly separable, meaning it can classify data points that can be separated by a straight line.

7.2. Structure of a Perceptron

1. **Inputs (Φ):** These are the features or data points fed into the perceptron. For example, in determining whether an email is spam or not, inputs could be the presence of certain words.
2. **Weights (ω):** Each input is multiplied by a corresponding weight that represents the input's importance.
3. **Bias (b):** A constant value added to adjust the output, enabling the model to fit the data better.
4. **Summation (∑):** The weighted inputs and bias are summed up.
5. **Activation Function:** This determines whether the neuron "fires" or not (e.g., outputs 0 or 1).

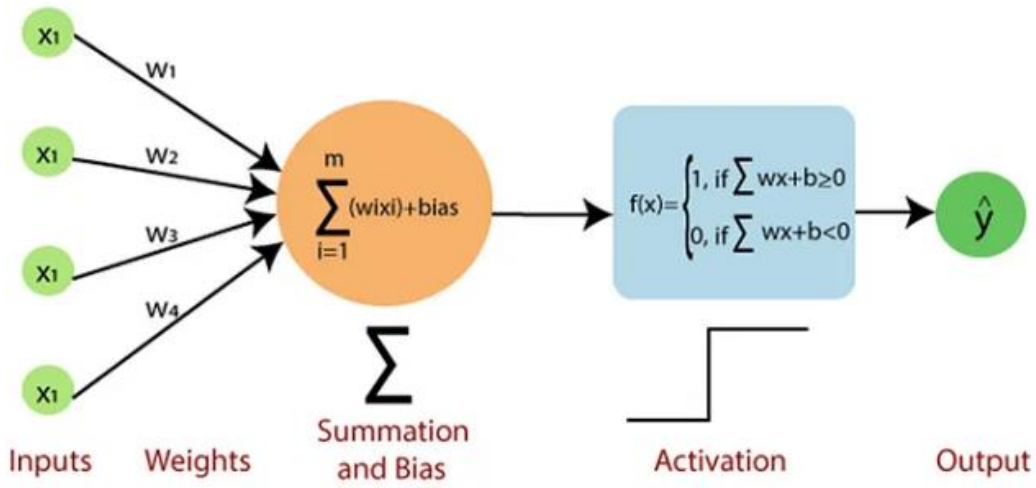


Fig: **Structure of a Perceptron**

7.3. Real-Life Example: Perceptron Model

Imagine you are creating a system to predict if a fruit is an apple (1) or not an apple (0). The inputs could be:

- : Is the fruit red? (Yes = 1, No = 0)
- : Is the fruit round? (Yes = 1, No = 0)

Weights are assigned based on how important each feature is. For example:

- : Being red is very important.
- : Being round is slightly less important.

If the weighted sum of these inputs plus a bias exceeds a threshold, the perceptron outputs 1 (apple); otherwise, it outputs 0 (not an apple).

---

8. Perceptron Learning Algorithm

The perceptron learning algorithm is used to train the perceptron by adjusting its weights and bias to minimize classification errors.

Steps of the Algorithm:

1. **Initialize weights and bias:** Start with small random values.
2. **For each training sample:**
    - Compute the perceptron's output .
    - Compare with the actual label (target output).
    - Update weights and bias if there is an error using:

Where:

- : Learning rate (a small positive number to control the speed of learning)
- : Target output (1 for correct classification, 0 otherwise)
- : Predicted output

3. **Repeat:** Iterate over the dataset until all samples are correctly classified or a maximum number of iterations is reached.

Real-Life Example: Email Spam Filter

Suppose you are designing a perceptron to classify emails as spam (1) or not spam (0). The inputs could include:

- : Does the email contain the word "free"? (Yes = 1, No = 0)
- : Is the email sent from an unknown sender? (Yes = 1, No = 0)

1. Initialize weights (e.g.,) and bias (e.g.,).
2. For an email with inputs and labelled as spam (), calculate the output:

If , update weights and bias using the learning rule. For example, if :

Repeat this process for all emails in the dataset until the perceptron classifies them correctly.

---

9. Error Correction

Definition:
Error correction is a process used in neural networks to minimize the difference between the predicted output of the model and the actual desired output. This difference is called the "error." The network learns by adjusting its parameters (weights and biases) to reduce this error.

Key Concept:

1. **Error Calculation:** The error is calculated as the difference between the actual output (target value) and the predicted output by the network.

$$\text{Error} = \text{Target Output} - \text{Predicted Output}$$

2. **Learning Process:** The network updates the weights and biases in such a way that the error is minimized. This process is iterative and continues until the error reaches an acceptable level.

Real-Life Example:
Spam Email Filtering:

- Suppose you create a neural network to classify emails as "Spam" or "Not Spam."
- Initially, the model might incorrectly classify some spam emails as "Not Spam." This is an error.
- The error correction mechanism adjusts the model's weights to better identify patterns in spam emails (like specific words or phrases). Over time, the network becomes better at distinguishing spam emails from non-spam ones.

For instance:

- Target Output: Spam (1)
- Predicted Output: Not Spam (0.4)
- Error: 1−0.4=0.61 - 0.4 = 0.61−0.4=0.6
- The network will reduce this error by adjusting its weights using a learning algorithm.

---

10. Gradient Descent Rules

Definition:
Gradient descent is an optimization algorithm used to minimize the error (or loss) in a neural network by adjusting the weights and biases. The goal is to find the optimal values for the parameters that result in the lowest error.

Key Concept:

1. **Gradient:** The gradient is the derivative of the error with respect to the weights. It indicates the direction and rate of change of the error.
2. **Descent:** The weights are updated in the direction opposite to the gradient to minimize the error.

The weight update rule is given by:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial E}{\partial w}$$

Where:

- w_{new}: Updated weight
- w_{old}: Current weight
- η: Learning rate (a small constant determining the step size)
- $\frac{\partial E}{\partial w}$: Gradient of the error with respect to the weight

---

Real-Life Example:
Self-Driving Cars:

- A neural network in a self-driving car learns to predict steering angles based on road images.
- Initially, the car might predict a wrong angle, leading to deviations from the lane. This deviation represents the error.
- Gradient descent updates the weights in the neural network to improve the predictions. Over time, the car learns to predict accurate steering angles, reducing lane deviations.

For instance:

- If the car steers too much to the right, the error gradient will point toward reducing the "right-steering" weights.
- The model adjusts its weights slightly in each iteration, moving closer to the optimal configuration for accurate driving.

UNIT II

BACKPROPAGATION NETWORKS

1. Introduction:

Backpropagation (short for "Backward Propagation of Errors") is a learning algorithm used in artificial neural networks (ANNs) to improve their accuracy. It helps the network learn from mistakes by adjusting its weights and biases to minimize errors.

Think of it like this:

- Imagine you are learning to throw a basketball into a hoop.
- The first few times, you might miss.
- But each time you adjust your throw—maybe you aim higher or use less force.
- Eventually, you get better at making the shot.
- Backpropagation helps a neural network learn in a similar way by adjusting its internal parameters to improve accuracy.

1.1. Why is Backpropagation Important?

Before backpropagation, training neural networks was slow and inefficient. With backpropagation:

- Neural networks can **learn faster** and improve their predictions.
- Errors can be **corrected automatically** in a structured way.
- It allows deep learning models (like those used in **speech recognition, self-driving cars, and image recognition**) to function efficiently.

1.2. How Does Backpropagation Work?

Backpropagation is based on two main steps:

1. **Forward Propagation:**
    - Input data passes through the network, layer by layer, until it reaches the output layer.
    - The network makes a prediction.
    - Example: A neural network for digit recognition might see a handwritten "5" and predict it's a "3" instead.
2. **Backward Propagation (Error Correction):**
    - The network calculates the error by comparing the predicted output with the correct output.
    - It then adjusts the weights of the connections to reduce the error using a method called **Gradient Descent** (which helps find the optimal values).
    - This process repeats until the network learns to make accurate predictions.

1.3. Real-Life Example of Backpropagation

Example 1: Learning to Ride a Bicycle

- At first, when you try to ride a bicycle, you might fall or wobble.
- Your brain **analyses the mistake** (error) and makes adjustments (like shifting balance, adjusting pedalling speed).
- Over time, with practice and corrections, you improve.
- This is similar to how backpropagation works—**it helps a neural network adjust and improve based on errors**.

Example 2: Virtual Assistants (Siri, Alexa, Google Assistant)

- When you talk to a virtual assistant, it sometimes misunderstands your words.
- If you correct it (for example, by choosing the right option or repeating yourself), it **learns from that mistake**.
- The next time you say something similar, it gets better at understanding.
- Backpropagation helps these systems **improve speech recognition accuracy** over time.

---

2. Supervised Learning with Neural Networks

2.1. What is Supervised Learning?

Supervised learning is a type of machine learning where a model learns from labelled data. This means that for every input, there is a correct output already given, and the model learns to map inputs to outputs by finding patterns in the data.

- **"Supervised" means "guided or taught"**—just like how a teacher corrects students' mistakes in class.
- The computer is **trained with examples**, and it learns from the mistakes using feedback.

2.2. How Does Supervised Learning Work?

Supervised learning works in three main steps:

1. **Training Phase:**
    - The model is given a dataset with **inputs (features)** and **correct outputs (labels)**.
    - The neural network makes predictions and compares them with the actual outputs.
    - If the prediction is wrong, the network **adjusts its weights** to improve accuracy.
2. **Testing Phase:**
    - Once trained, the model is tested on new, unseen data to check how well it has learned.
3. **Prediction Phase:**
    - After training & testing, the model can make predictions on new data.

2.3. Real-Life Example: Learning to Recognize Handwritten Digits

Imagine you are training a neural network to recognize handwritten numbers (like a digit recognition system used by banks for reading cheques).

1. **Step 1: Training the Neural Network**
    - We show the model **thousands of images of handwritten numbers (0-9)**.
    - Each image has a **correct label** (e.g., a picture of '7' is labelled as '7').
    - The neural network **analyses patterns in the images** and learns to associate them with numbers.
2. **Step 2: Testing the Model**
    - Once trained, we test the model with new handwritten numbers it has never seen before.
    - If the model correctly identifies most numbers, it means it has **learned well**.
3. **Step 3: Real-World Use**
    - When you deposit a cheque at a bank, the ATM scans the handwritten amount and recognizes the digits correctly using a trained neural network.

2.4. How Neural Networks Learn in Supervised Learning?

Neural networks learn using an algorithm called **Backpropagation (Backward Propagation of Errors)**.

Key Steps in Learning:

1. **Forward Pass:**
    - The input goes through the neural network and produces an output (prediction).
2. **Error Calculation:**
    - The model checks how far its prediction is from the correct answer (error calculation).
3. **Backpropagation (Backward Pass):**
    - The error is sent **backward** through the network to adjust the weights.
    - This ensures that the network learns from its mistakes and improves.
4. **Weight Update using Gradient Descent:**
    - The network updates its weights using an optimization technique called **Gradient Descent** to minimize the error.
5. **Repeat the Process:**
    - Steps 1-4 are repeated **many times** until the model becomes very accurate.

2.5. Real-Life Example: Learning to Solve Math Problems

Think about how you learn to solve math problems:

1. **Step 1 (Training):**
    - Your teacher gives you practice problems with answers.
    - You try solving them and check if you got them right.

2. **Step 2 (Error Checking):**
    - If you made a mistake, your teacher points it out.
    - You understand what went wrong.
3. **Step 3 (Correction & Learning):**
    - You try solving similar problems again.
    - Over time, you **learn** how to get the correct answers without mistakes.

This is exactly how supervised learning works in neural networks!

2.6. Types of Supervised Learning Problems

There are two main types of supervised learning problems:

1. Classification (Predicting a category)

- The output is a category (e.g., Yes/No, Spam/Not Spam, Dog/Cat).
- Example: Gmail classifies emails as **Spam or Not Spam** based on previous emails labelled by users.

2. Regression (Predicting a number)

- The output is a continuous value (e.g., price, temperature, age).
- Example: A house price prediction system predicts the **price of a house** based on factors like size, location, and number of bedrooms.

---

3. Architecture of Backpropagation Network

A backpropagation network consists of three main layers:

1. **Input Layer**
2. **Hidden Layer(s)**
3. **Output Layer**

Each layer is made up of **neurons**, which are the fundamental processing units of the network.

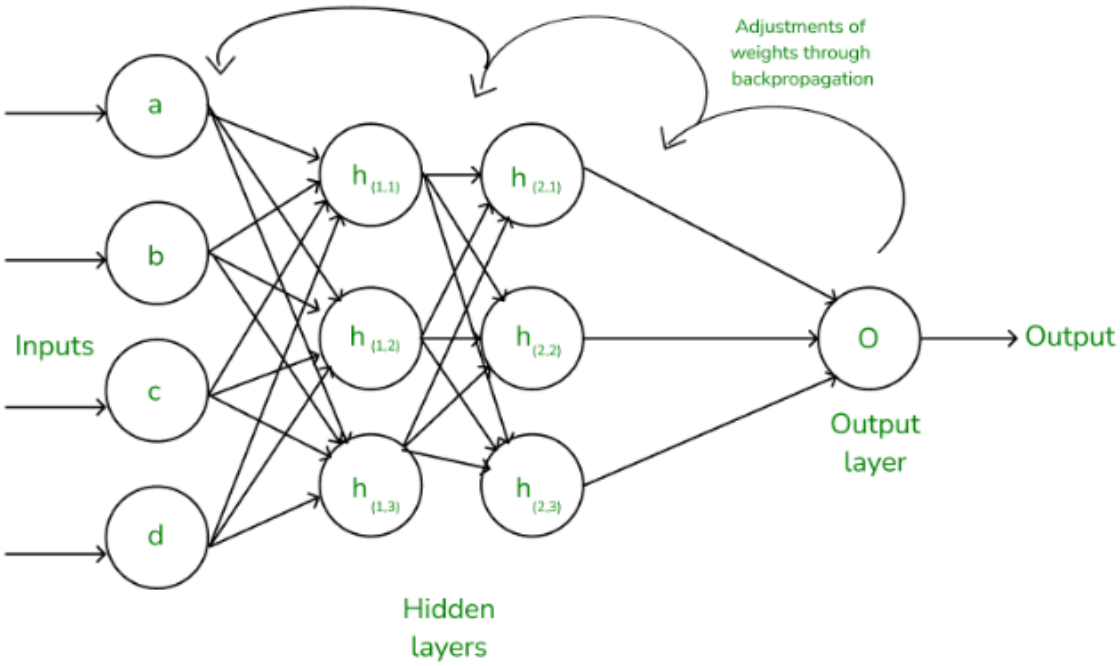


Fig: Architecture of Backpropagation Network

(i) Input Layer

- This is the first layer of the network.
- It receives raw data (features) from the external environment.
- Each input neuron represents one feature of the data.

❖ **Real-Life Example:**
Think of an **image recognition system** that classifies images as either a "Dog" or "Cat".

- The input layer takes pixel values of an image as input.
- If the image has 100 pixels, the input layer will have 100 neurons (one for each pixel).

(ii) Hidden Layer(s)

- These layers process the input data by performing mathematical operations.
- The more hidden layers a network has, the **deeper** it is (Deep Learning).
- Each neuron in the hidden layer applies an **activation function** to decide whether to pass the information forward.

❖ **Real-Life Example:**
Continuing with the **image recognition** example:

- The hidden layers extract patterns from the image, such as edges, colours, and shapes.
- The deeper the network, the better it understands complex features (like eyes, nose, ears of a cat or dog).

(iii) Output Layer

- This is the final layer that gives the result.
- The number of neurons in the output layer depends on the **type of problem**:
  - **Binary classification (Yes/No, Dog/Cat)** → 1 neuron
  - **Multi-class classification (Car, Bike, Truck, Bus)** → More than 1 neuron

❖ **Real-Life Example:**
For the **dog vs. cat classifier**:

- If the output is **1**, it means the image is a **dog**.
- If the output is **0**, it means the image is a **cat**.

3.1. Working of Backpropagation Algorithm (Step-by-Step)

The backpropagation network works in two phases:

1. **Forward Propagation** → The input data passes through the network, and the output is calculated.
2. **Backward Propagation (Error Correction)** → The network adjusts its weights to reduce errors.

Step 1: Forward Propagation

- The input data is passed through each layer.
- The neurons in each layer apply weights and activation functions.
- The final output is generated.

❖ **Example:**
If you give an image of a **dog**, the network will predict whether it is a dog or not.

Step 2: Compute Error

- The network compares the predicted output with the actual output.
- The difference between them is called the **error (loss)**.

❖ **Example:**
If the network predicted "Cat" but the actual image was a "Dog", then an **error** is generated.

Step 3: Backward Propagation (Error Correction)

- The error is propagated backward to adjust the weights.
- The **Gradient Descent Algorithm** is used to minimize the error step by step.

✓ **Example:**
Imagine a **teacher correcting a student's mistakes**:

- If a student writes **wrong answers** in an exam, the teacher marks them incorrect.
- The student **learns from mistakes** and corrects them in the next test.
- Similarly, the network **updates its weights** and improves its accuracy.

3.2. Key Features of Backpropagation Network

✓ **Supervised Learning** – It learns from labelled data (i.e., input data with correct answers).
✓ **High Accuracy** – Used in deep learning to solve complex problems.
✓ **Error Minimization** – Continuously reduces error with each training step.

3.3. Real-Life Applications of Backpropagation Networks

1. **Face Recognition (Unlocking Phones with Face ID)**
   - The network learns to recognize facial features.
   - It compares the detected face with stored images and grants access if it matches.
2. **Self-Driving Cars**
   - Neural networks process sensor data (camera, radar, LiDAR).
   - It learns to recognize road signs, detect obstacles, and drive safely.
3. **Chatbots & Virtual Assistants (Siri, Alexa, Google Assistant)**
   - Uses backpropagation to learn from past conversations.
   - Improves response accuracy over time.

---

4. The Perceptron Model

4.1. Introduction to Perceptron

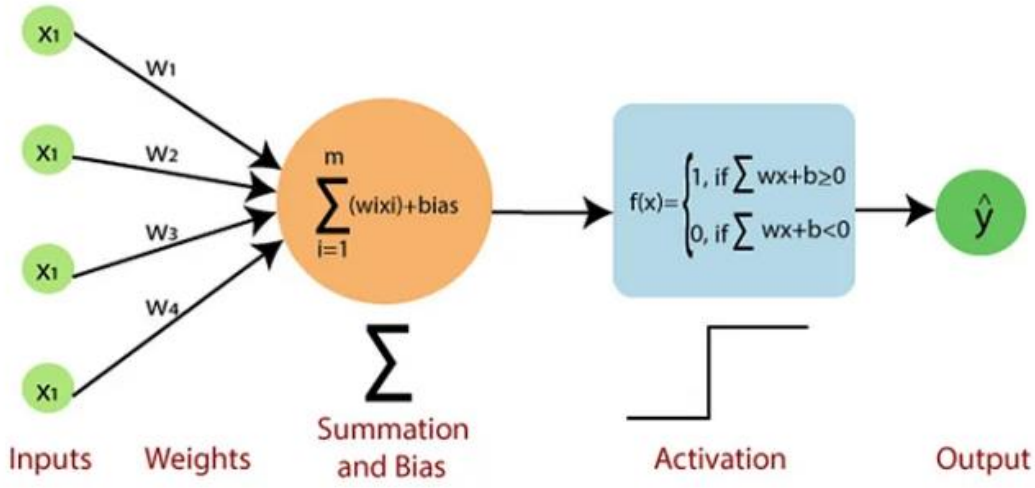
A **perceptron** is the simplest type of artificial neural network. It is a type of **supervised learning model** used for classification tasks. A perceptron works like a decision-making system that can determine whether something belongs to one category or another (for example, "spam" or "not spam" in email classification).

The perceptron was invented by **Frank Rosenblatt** in 1958 and is the foundation of modern neural networks.

4.2. Structure of a Perceptron

A perceptron is made up of the following components:

1. **Input Layer**: This consists of multiple input values (features) that provide information to the perceptron.
2. **Weights (W)**: Each input is assigned a weight that determines its importance.
3. **Summation Function (Σ)**: The weighted sum of all inputs is calculated.
4. **Activation Function**: The weighted sum is passed through an activation function that decides whether the neuron should "fire" (output a value) or not.
5. **Output**: The final result, which is either **0 (inactive)** or **1 (active)** based on the activation function.



Mathematical Representation:

$y=f(\sum(w_i \cdot x_i)+b)$

Where:

- xi = Inputs
- wi = Weights

- b = Bias (adjusts the decision boundary)
- f = Activation function
- y = Output

4.3. Working of a Perceptron

A perceptron makes decisions using a **step function** as an activation function.

- If the weighted sum is greater than a certain threshold, the perceptron outputs **1 (Yes/True/Positive)**.
- If it is less than the threshold, the perceptron outputs **0 (No/False/Negative)**.

Example:

Imagine a **student grading system** where a teacher decides whether a student **passes or fails** based on two factors:

1. **Assignment Marks (X1)**
2. **Exam Marks (X2)**

The decision is based on the formula:

$$y=f(w1\cdot X1+w2\cdot X2+b)$$

- If the final result is **above the threshold**, the student **passes (1)**.
- If the result is **below the threshold**, the student **fails (0)**.

This is how a perceptron makes binary decisions!

4.4. Real-Life Example of a Perceptron

Example 1: Spam Email Detection

A perceptron can classify emails as **Spam (1) or Not Spam (0)** based on different features:

Feature	Spam Email (1)	Not Spam (0)
Contains "Free Money"	Yes (1)	No (0)
Contains "Urgent"	Yes (1)	No (0)
Contains "Dear Customer"	Yes (1)	No (0)
Contains "Meeting Invitation"	No (0)	Yes (1)

- If the weighted sum of these features exceeds a threshold, the email is classified as **Spam (1)**.
- If it does not exceed the threshold, it is classified as **Not Spam (0)**.

This is how perceptrons help in real-world applications!

4.5. Limitations of a Perceptron

While perceptrons are useful, they have some limitations:

1. **Cannot Solve Complex Problems**: Perceptrons can only solve **linearly separable problems** (like distinguishing cats vs. dogs), but they **fail** at problems requiring multiple layers (like recognizing handwritten digits).
2. **Single Layer Network**: A simple perceptron cannot handle **nonlinear** problems like the XOR problem.
3. **No Hidden Layers**: More complex decisions require deeper networks with hidden layers.

To overcome these limitations, **multi-layer perceptrons (MLPs) and backpropagation** are used in modern neural networks.

---

5. Single Layer Artificial Neural Network

A **Single Layer Artificial Neural Network (ANN)** is the simplest type of neural network that consists of only one layer of neurons (also called nodes or perceptrons). This type of network is mainly used for simple classification problems where data can be separated into different categories.

Structure of a Single Layer ANN

A single-layer neural network consists of three main components:

1. **Input Layer:** Receives input data (features).
2. **Weights:** Each input is multiplied by a weight, which determines its importance.
3. **Summation Function:** Adds up all the weighted inputs.
4. **Activation Function:** Decides whether a neuron should be activated or not, based on a threshold.
5. **Output Layer:** Produces the final result (prediction).

Mathematical Representation

If we have inputs x1,x2,x3,..., xn and corresponding weights w1,w2,w3,..., wn, the neuron processes them as:

$$y = f(w_1x_1 + w_2x_2 + \ldots + w_nx_n + b)$$

where:

- y is the output,
- w are the weights,
- x are the inputs,
- b is the bias,
- f is the activation function.

Real-Life Example: Email Spam Filter

Imagine you receive an email, and you want to classify it as **Spam or Not Spam** using a single-layer neural network.

1. **Inputs:** Features of an email, such as:
    - Presence of the word “free” (x1)
    - Presence of “lottery” (x2)
    - Number of capital letters (x3)
    - Presence of a link (x4)
2. **Weights:** Each input has a weight representing its importance.
    - Example: The word “lottery” (x2) might have a higher weight because spam emails often contain this word.
3. **Summation Function:** Computes the weighted sum of inputs.

$$z = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

4. **Activation Function:**
    - If z exceeds a certain threshold, classify the email as **Spam (1)**.
    - Otherwise, classify it as **Not Spam (0)**.

Thus, the neural network helps in classifying emails as spam or not spam based on the learned patterns.

Limitations of Single-Layer ANN

- Can only handle simple problems where data can be separated with a straight line (Linear Separation).
- Cannot solve complex problems like recognizing handwritten digits or face recognition.

---

6. Multilayer Perceptron (MLP)

A **Multilayer Perceptron (MLP)** is a type of artificial neural network (ANN) that consists of multiple layers of neurons. It is widely used in tasks like image recognition, speech recognition, and predicting stock prices.

MLP is a **supervised learning** model, meaning it learns from labelled data, where the correct output is known during training.

Structure of a Multilayer Perceptron (MLP)

An MLP consists of three main types of layers:

1. **Input Layer** – This layer receives the input data (features). Each neuron in this layer represents one input feature.
2. **Hidden Layers** – These layers process the input data and learn patterns. An MLP can have one or more hidden layers.
3. **Output Layer** – This layer gives the final result, such as classification (e.g., identifying a cat vs. dog) or regression (e.g., predicting house prices).

Each neuron in one layer is connected to every neuron in the next layer through **weights**. These weights determine the importance of each connection.

Working of Multilayer Perceptron (MLP)

Step 1: Forward Propagation

- The input data passes through the network from the input layer to the output layer.
- Each neuron processes the input using the **weighted sum formula** and applies an **activation function** to introduce non-linearity.
- This allows MLP to learn complex patterns.

Mathematically, a neuron's output is calculated as:

$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + ... + b)$$

where:

- $x_1, x_2, x_3, ...$ are input values
- $w_1, w_2, w_3, ...$ are weights
- b is the bias term
- f is the activation function

Step 2: Error Calculation

- The output of the network is compared with the actual result (ground truth), and an **error (loss)** is calculated.
- The goal of training is to **reduce this error** by adjusting the weights of the network.

Step 3: Backpropagation

- The error is propagated backward through the network.
- The weights are updated using an optimization algorithm like **Gradient Descent** to reduce the error.
- This process is repeated for multiple iterations until the network learns the correct patterns.

Real-Life Example of Multilayer Perceptron (MLP)

Example 1: Handwriting Recognition (Digit Recognition)

Imagine you want to create a system that can recognize handwritten digits (0-9). Here's how an MLP can be used:

1. **Input Layer:** The system takes an image of a handwritten digit (say "7"). The image is converted into pixel values, which serve as inputs.
2. **Hidden Layers:** The network processes these pixel values and extracts important patterns, such as edges and curves.
3. **Output Layer:** The network predicts the digit as "7". If the prediction is incorrect, backpropagation adjusts the weights to improve accuracy.

Over time, the MLP learns to recognize handwritten digits correctly with high accuracy.

Example 2: Spam Email Detection

Consider an email filtering system that detects spam emails.

1. **Input Layer:** The system takes an email as input. It extracts features like the number of spam words (e.g., "win", "free", "lottery").
2. **Hidden Layers:** The network processes these features and identifies patterns common in spam emails.
3. **Output Layer:** The system classifies the email as **Spam (1)** or **Not Spam (0)**. If the classification is wrong, the network adjusts its weights through backpropagation to improve future predictions.

---

7. Backpropagation Algorithm

7.1. What is Backpropagation?

Backpropagation (short for *backward propagation of errors*) is an algorithm used to train artificial neural networks by adjusting the weights based on the error in predictions.

How It Works (Step-by-Step Process)

1. **Forward Propagation:** The input is passed through the network layer by layer to generate an output.
2. **Error Calculation:** The difference between the predicted and actual output is calculated (Loss Function).
3. **Backward Propagation:** The error is propagated backward to adjust the weights.
4. **Weight Update:** The weights are updated to minimize the error.

Real-Life Example:

🔻 *Google Translate*

- Suppose you type “Hello” and the translation is incorrect.
- The system compares its translation to correct translations and adjusts the model.
- Over time, it improves by learning from mistakes using backpropagation.

7.2. Backpropagation Learning

How Does Backpropagation Help in Learning?

- Backpropagation helps the neural network **"learn"** from mistakes.
- It corrects the weights based on feedback.
- The learning continues until the network achieves high accuracy.

Real-Life Example:

🔻 *Self-Driving Cars*

- A car must recognize stop signs and pedestrians.
- If it incorrectly identifies an object, backpropagation helps correct the mistake.
- After several corrections, the car learns to drive safely.

7.3. Training of the Network

Why is Training Needed?

Training a neural network means adjusting its weights so that it can make accurate predictions on new data.

Steps in Training:

1. Provide input data.
2. Compute the output.
3. Compare it with actual output.
4. Adjust weights using backpropagation.
5. Repeat until accuracy improves.

Real-Life Example:

🔻 *Face Recognition on Smartphones*

- When setting up face unlock, the phone trains itself to recognize your face.
- It adjusts its internal network to match your unique facial features.
- The more images it sees, the better it gets at recognizing you.

7.4. Calculation of Error

Why is Error Calculation Important?

Error calculation helps the network understand how far its predictions are from the actual values. The smaller the error, the better the model.

Mathematical Formula:

$$\text{Error} = \text{Actual Output} - \text{Predicted Output}$$